

Chop Chop Serve

Game Design Document

Executive Summary	3
<i>Game concept overview</i>	3
<i>Target Audience</i>	3
<i>Platform & Technical Requirements</i>	3
<i>Develop Timeline Estimate</i>	3
Game Overview	4
<i>Core gameplay mechanics</i>	4
<i>Genre & Style</i>	4
<i>Unique Selling Points</i>	4
<i>Player Experience goals</i>	4
Gameplay Mechanics	5
<i>Core Game Loop</i>	5
<i>Player Actions and Controls</i>	5
<i>Game Systems and Interactions</i>	5
<i>Progression and Rewards</i>	5
Technical Design	6
<i>Engine Choice</i>	6
<i>Technical requirements and architecture</i>	6
<i>Performance targets and optimization</i>	6
<i>Platform considerations and deployment strategy</i>	6
Art and Audio Direction	7
<i>Visual Style and Art Direction</i>	7
<i>Audio design and music style</i>	7
<i>UI/UX</i>	7
Level Design	8
<i>Game world structure</i>	8
<i>Level Progression</i>	8
Production Plan	10
<i>Development Phase and milestones</i>	10
<i>Resource management and team structure</i>	10
<i>Risk assessment and mitigation strategies</i>	10
<i>Version control and project management strategy</i>	10
<i>Testing and quality assurance</i>	10

Executive Summary

Game Concept Overview

'Chop Chop Serve' is a chaotic, competitive cooking game where two restaurants compete head-to-head to be the last one standing. Players take the role of a rookie entrepreneur running a local restaurant in the heart of the city. With your passion for food, you cook and serve hungry customers, each day becoming more challenging from more complex dish recipes and impatient customers to suspiciously timed disruptions.

With a new restaurant start-up near your restaurant, you both plan to sabotage each other's restaurant to drive each other out of business. It's a race against time to see who has the better restaurant.

Target Audience

We aim to target players who enjoy existing cooking games like 'Overcooked' and 'PlateUp!'. This target audience felt suited for our game as it's a chaotic cooking game with similar mechanics and systems to existing games in the genre. With the nature of the game being local multiplayer, we also want to target players who enjoy party games similar to 'Moving Out' and 'Overcooked'. This refined audience will help inform decisions towards the game's development.

Platform & Technical Requirements

Ideally, the game will be released on Windows and will be uploaded to Itch.io and Steam if the game is in a good spot. If we have time, it would be nice to explore the possibility of console support, but this would only be an option once we have a stable Windows version.

We want the game to be able to be played on low-end pc's and above. This is mainly so the game is accessible to as many people as possible. With the game not being low poly and not having big environments, there shouldn't be any reason why it shouldn't be able to run on low-end pc's.

Develop Timeline Estimate

We plan to work on this project for **3 months** from **[July 23rd, 2025]** to **[October 25th, 2025]**.

Game Overview

Core Gameplay Mechanics

Move:

Players are able to traverse their restaurant.

Sabotage:

Players can choose to spend their money to sabotage the opposing restaurant.

Upgrade:

Players can choose to spend their money to upgrade their own restaurant. This is so you can make your restaurant more efficient at cooking meals to earn more money.

Interact:

Players will be able to interact with items on the floor and appliances.

Dash:

Players will be able to dash around in the direction they are moving (if standing still in the direction they are facing).

Genre & Style

We are aiming to have this game fit into the **Cooking Simulation** and **Party Game** genres. This genre of games is filled by most notably 'Overcooked' and 'Plates Up!'. We want to find our area in this genre by exploring the various aspects of cooking. The style of gameplay will be quite similar to these games to keep players familiarized and easier to onboard if they have already played similar games.

Unique Selling Points

There are three main USP's we have that separate our game from the rest of the genre.

Sabotages:

This system allows teams to be able to send sabotage to the other restaurant. This idea came from an already existing game, 'BTD Battles', with its sabotage system of sending bloons to the other player. We want to implement this into our game with you being able to spend money to send sabotages to hinder the other team's progress, such as **Power Cuts**, **Food Critics**, and **Rat Swarm**.

Upgrading:

Players can spend money on upgrading their restaurant. This allows players to "freely" customize the layout of their restaurant and build it up how they envision it. This system has been explored in-depth in 'Plates Up!' but was in the context of a more co-operative setting. We are using this upgrade system for player agency, choosing between using their money to upgrade or to sabotage.

Versus:

The concept of two separate restaurants versus each other hasn't been the focal point of a cooking game. 'Overcooked 2' has a versus mode where you have to join two restaurants together and can sabotage them by grabbing items and standing in the way of them. We want to explore this with our twist by having the restaurants split and using the sabotage system to hinder the progress of the other team.

Player Experience Goals

During the prep phases, players should feel a moment of calm and control. This is the time for players to breathe, strategize, and make the choices they need to make. Once the serving phase starts, the tone will shift. Players will feel the intense stress of needing to micro-manage, cooking, serving, and sabotaging.

Gameplay Mechanics

Core Game Loop

Prepare → Serve → Strategize → Repeat

Prepare:

Players will **prepare** food for the order they are working on completing.

Serve:

Once players have completed the order they will **serve** this to the customer and earn money.

Strategize:

With this money, players can **strategize** whether to: Upgrade their restaurant; Sabotage the opposing restaurant or save their money for a future purchase.

Repeat:

This loop repeats with players needing to consistently prepare, serve and strategize.

Player Actions and Controls

Walk:

Players will be able to traverse around the restaurant to efficiently navigate. This is used with the traditional WASD movement. This is the slower movement option compared to **dash**, but it won't affect reputation and interactions with other players.

Interact:

Objects within the restaurant will be able to be interacted with. Such as customers, appliances, and food. This can vary from picking the item up, handing a customer their meal, or placing the item held in your hand on an appliance. This interaction system gets a more in-depth explanation in the [Interaction System] section.

Throw:

When the player is holding an item, they can hold the keybind down, and when they let go, they will throw the item in the direction the player is facing. While holding the keybind down, it will charge the strength of the throw and will cap at a certain distance.

Action:

It's a bit of a special use case depending on the task and/or item held by the player. Various items will have a special action that can be used. An example of this is the fire extinguisher; you can pick it up, but to shoot the foam requires you to use the **action** control. Tasks like rerouting power after a power cut will use this action as well to hit the skill checks. Upgrading will require using the **action** to confirm an upgrade.

Dash:

Propels the player forward in the direction of their velocity. If standing still, it will propel in the direction the character is facing. This is so players can efficiently get around the kitchen faster than just walking around. There will be a small delay between dashes so players don't fully spam dashing. The trade-off will be that bumping into a customer mid-dash has the chance to lose reputation, and if bumped into, your teammate will have the chance to drop what they are holding.

Control Scheme		
<i>Control</i>	<i>Keyboard</i>	<i>Gamepad</i>
Walk	W / A / S / D	Left Joystick
Interact	E	X
Throw	Left Shift	L2
Action	Q	R2
Dash	Space	A

Game Systems and Interactions

Phase System:

The gameplay is broken into two phases. **Prep Phase** and **Serving Phase**. Prep phases last 30 seconds and serve to give the player time to assess whether to sabotage, upgrade or prep for the next round. It will juxtapose between the serving phase and will allow players breathing room. The serving phase is where players will need to serve customers. Each day will increase

the amount of customers and will fall back on a time based system if the player is stalling how long they are taking to serve. The game will start on a prep phase. Then will toggle between serving and prep phases

Reputation System:

The reputation will be separate between the teams. This serves as the restaurant's health. Your reputation reaching zero, results in your team losing. The main goal of the game is to keep your reputation as high as possible.

Reputation can be lost by:

- Failing to serve customer
- Getting a customer's order wrong
- Leaving a sabotage unattended for too long
- Not cleaning plates from tables

Reputation can be gained by:

- Correctly serving a customer
- Upgrading restaurant

Reputation will be shown via a UI on top of the screen and is visible by both teams. This is so players know the reputation of the other team and can strategize their next move.

Sabotage System:

During both **the Prep Phase** and **the Serving Phase**, players will be able to sabotage the other restaurant by using their team's money. Sabotages are a way to increase tension between the other team, but they need to manage serving their restaurant while stopping the sabotage. Sabotages cannot be used if the other team is in their **Prep Phase**.

[Crate Switch | Cost: 400 | Lasts: 15s] - All food crates will change the ingredient they give and will display a question mark instead of the usual ingredient symbol. After the duration, all food crate symbols will return to their correct symbols, and the ingredient they give will stay the same post-sabotage.

[Water Spill | Cost: 450 | Lasts: 10s] - Random small areas of the ground will become covered in water. If a player walks over this area, they will walk more slowly. If they dash while in this area, they will drop the item they are holding. If a customer walked over this water spill, there's

a 10% chance the customer falls over and the team will lose 3 reputation points.

[Fire | Cost: 600 | Lasts: ∞] - The stove with the most progress will be set on fire. If left unattended, it will spread to adjacent benches. The fire effect will cause exponential cooking of whatever item is on the surface, and in some cases, will disable the appliance, which will then need to be repaired. If left for a prolonged amount of time, it will trigger a fire alarm, causing all customers to leave the restaurant and end the day early. Using a fire extinguisher foam on a fire will extinguish it.

[Food Critic | Cost: 700 | Lasts: ∞] - Will convert the first customer waiting in the queue into a Food Critic.

[Rat Swarm | Cost: 900 | Last: 20s] - Rats will spawn every 2 seconds from random grates in the kitchen and roam to the closest bench that has an item. Rats cannot target the same item. Once an item is grabbed, the rat will run towards the grate and will disappear with the item in hand. Rats are susceptible to rat spray, as being sprayed will kill them, dropping the item they are carrying. After a duration, the rats will stop spawning.

[Power Cut | Cost: 1200 | Last: 15s] - All appliances that use power will stop working. After the duration they will turn back on.

Interaction System:

Players will be able to interact with two main categories of interactables: Items and Appliances. When near something that is interactable, a light overlay glow will appear on the item or appliance to signify to the player that they can interact with it.

Items, when interacted with, will be equipped in the player's hands. These can be from food crates that give items, items that are on the floor, or appliances that have items on top of them. If the player interacts with an item while already having an item present in their hand, it will drop the item and replace it with the item they just interacted with.

Appliances store items on top of or within them when interacting with a valid item in the player's hand. When the player has no item in hand, it will remove the item that's on the appliance and put it in their hand

Currency System:

Players are able to earn money from serving customers. Each team will have its own money, which will accumulate from primarily serving customers. Money can be used during both the Prep and Serving phases, And is used to either upgrade your restaurant's appliances, or to sabotage the other restaurant. There won't be a maximum amount of money a restaurant can have.

Appliance System:

Appliances are physical objects in the restaurant that either store or change the state of the food; Either by cooking, mixing, or frying. This system purely handles the state changes of the food. An example would be having a tomato on a chopping board. When it is interacted with, it will turn into chopped tomatoes. It knows what ingredients and their states are valid to be placed on the appliance and can change their state after a certain time. The system for combining foods is handled by the cooking system.

Building System:

There will be an appliance called the "Crafting Bench" that players are always given at the start of the game. When interacted with, an in-world UI will be visible showing a list of all the appliances that are available for the player to buy. Each appliance will cost money to buy and is locked behind a certain amount of reputation. Once an appliance is brought, the cost of it will scale up and the player will have a present in their hands. Players can drop this present to build the appliance they brought. There will be an outline of the appliance so the players know where it will be built before dropping. The kitchen is laid out in a grid-like manner to make it more predictable where appliances will be placed. The appliance will conform to this grid when building.

Upgrade System:

Players will be able to pick up a hammer, which will be used to upgrade certain appliances. While holding the hammer, the player can walk up to an appliance, and an in-world UI will show up to show how much the upgrade will cost and the effects of the upgrade. Once a player interacts with the appliance, it will upgrade it with confetti and will deduct the amount from the restaurant's currency.

Customer System:

Customers will spawn with random attire during the **Serving Phase** and will spawn outside the camera's view and walk up towards the front door of the restaurant. They will wait in a queue in front of the restaurant until there's a free seat at a table. This queue will have a max of five

customers; no more customers will spawn until a customer leaves the queue. Once seated at a table, they will have a delay between **10 and 15** minutes before they order food. Customers will always order meals in the order:

Starter → Main → Dessert

Customers won't always order all three. Meaning a customer may only order a starter and then leave after being served, a customer may order all three and leave after the dessert, or a customer may only order dessert. These orders will happen subsequently after another, so a customer may order a starter, then once they get served their starter, they will have the **10 - 15 second** delay again and order their main. In the case the player serves all the customers' orders on time, they will gain money and reputation.

Then the customer will vacate their seat and leave the restaurant. If a customer doesn't get served their order in time, the player will gain no money and lose reputation. The customer will vacate their seat and leave the restaurant. Customers, such as those 20%, become irritated if the player dashes into them while not in a seat. This situation will decrease the reputation by a **default amount of 2**. This is so the player is more mindful of when they dash. Once the team **reaches 10** reputation, they will not lose reputation from dashing into customers. This is so the player doesn't lose to dashing into a customer, as this feels like an unawarded win to the other team.

There are special types of customers called **Food Critics; these** will appear once per day between 50% - 100% of each serving day. These customers will have more emphasis on getting their order perfectly cooked. They will give you more money and reputation if you serve good quality food, but will lose **a lot** of reputation if messed up. These critics have a shorter span of waiting for their food.

Order System:

Customers, after being seated at a table, will request a meal from the available meals on the menu. There will be a chance for variations of these meals that a player will need to account and prepare for, such as vegan options and allergies. This keeps each order less repetitive for the player, but also has that familiarity with the meals. Orders will last **40s** and if the order is not fulfilled, the team will lose reputation deducted by the calculation:

$$(10 + \text{DayNum} * 5)$$

This is so the players lose much during the starting days but further the games progress the more risk and tension will build up to complete an order.

Movement System:

The player is able to traverse around the map in a 3D manner. Players will be able to dash to propel themselves in the direction they are moving. This allows players to efficiently move around the restaurant.

Cooking System:

This system involves how foods and items are combined to create new foods. With the appliance system handling the cooking state of the food. This only handles combining food. An example of this is how a pot of water and three chopped mushrooms, when cooked on a stove, will make mushroom soup. The appliance only cooks it, but the cooking system handles what it wants to change into.

Dishes:

Dishes are decomposed into three categories: Starters, Mains, and Desserts. Each one will have its standard dish, with some having variants of the dish. An example would be the Soup having three variants of the dish [Mushroom, Tomato, Onion]. Dishes with [Standard] have no variants.

Starters

- Bread [Normal, Cheese, Garlic]
- Soup [Mushroom, Tomato, Onion, Pumpkin]
- Mac & Cheese [Standard]
- Onion Rings [Standard]

Mains

- Pizza [Cheese, Meat, Hawaiian]
- Burger [Beef, Chicken, Fish]
- Steak [Chips, W/O Chips]
- Bolognese [Standard]
- Fish & Chips [Standard]
- Tacos [Mushroom, Beef, Chicken]

Deserts

- Pie [Apple, Pumpkin, Pineapple]
- Ice cream [Strawberry, Chocolate, Vanilla]

- Brownies [Standard]
- Pancakes [Standard]

Recipes:

Examples of recipes

Bread (Normal)

Ingredients: *Flour, Water*

Items: *Pot, Tray, Whisk, Plate*

Appliances: *Sink, Oven*

Process:

- Fill pot with water
- Pour flour into pot
- Grab whisk and interact with pot until mixed
- Place contents from pot into a tray
- Place tray into oven until cooked
- Interact with oven to grab tray
- Interact with plate with tray

Bread (Cheese)

Ingredients: *Flour, Water, Cheese*

Items: *Pot, Tray, Whisk, Plate*

Appliances: *Sink, Oven*

Process:

- Fill pot with water
- Pour flour into pot
- Grab whisk and interact with pot until mixed
- Place contents from pot into a tray
- Grate cheese
- Grab tray and interact with grated cheese
- Place tray into oven until cooked
- Interact with oven to grab tray
- Interact with plate with tray

Bread (Garlic)

Ingredients: *Flour, Water, Garlic*

Items: *Pot, Tray, Whisk, Plate*

Appliances: *Sink, Oven*

Process:

- Fill pot with water

- Pour flour into pot
- Grab whisk and interact with pot until mixed
- Place contents from pot into a tray
- Grate Garlic
- Grab tray and interact with grated garlic
- Place tray into oven until cooked
- Interact with oven to grab tray
- Interact with plate with tray

Soup (Mushroom)

Ingredients: *Water, Mushroom*

Items: *Pot, Chopping Board, Bowl*

Appliances: *Sink, Stove*

Process:

- Fill pot with water
- Chop mushrooms 3x
- Grab pot and interact with chopped mushrooms 3x
- Place pot on stove until cooked
- Interact with stove to grab pot
- Interact with bowl with pot

Soup (Tomato)

Ingredients: *Water, Tomato*

Items: *Pot, Chopping Board, Bowl*

Appliances: *Sink, Stove*

Process:

- Fill pot with water
- Chop tomatoes 3x
- Grab pot and interact with chopped tomatoes 3x
- Place pot on stove until cooked
- Interact with stove to grab pot
- Interact with bowl with pot

Soup (Onion)

Ingredients: *Water, Onion*

Items: *Pot, Chopping Board, Bowl*

Appliances: *Sink, Stove*

Process:

- Fill pot with water
- Chop onion 3x

- Grab pot and interact with chopped onion 3x
- Place pot on stove until cooked
- Interact with stove to grab pot
- Interact with bowl with pot

Soup (Pumpkin)

Ingredients: *Water, Pumpkin*

Items: *Pot, Chopping Board, Bowl*

Appliances: *Sink, Stove*

Process:

- Fill pot with water
- Chop pumpkin 3x
- Grab pot and interact with chopped pumpkin 3x
- Place pot on stove until cooked
- Interact with stove to grab pot
- Interact with bowl with pot

Onion Rings

Ingredients: *Flour, Onion*

Items: *Fryer Basket, Plate*

Appliances: *Deep Fryer, Chopping Board*

Process:

- Chop onion at chopping board
- Grab Fryer Basket and interact with chopped onions
- Pour flour into fryer basket
- Place into Deep Fryer until cooked
- Interact bowl with deep fryer

Pancakes

Ingredients: *Flour, Water, Milk*

Items: *Plate, Pan, Whisk*

Appliances: *Stove, Sink*

Process:

- Fill pan with water
- Add milk to pan
- Add flour to pan
- Whisk contents of pan

- Place pan on stove until cooked
- Grab pan and interact with plate

Ingredients:

Tomato
Garlic
Mushroom
Cheese
Beef
Chicken
Pineapple
Strawberry
Cocoa
Milk
Flour
Fish
Onion
Water
Potato
Pumpkin

Appliances:

Bench:

Power: No

Items and ingredients can be placed on top of benches. Benches can store one item at a time. There are variants of benches that have appliances on top of them and are used for cooking and not for placing items.

Cutting Board:

Power: No

A variant of the bench that has a chopping board on top of it. Ingredients can be placed on this and can be interacted with to be chopped. Chopping takes 5 seconds to happen. Then the ingredients can be placed into a pot or pan.

Stove:

Power: Yes

Pots and pans can be placed on the stove, which will start to cook whatever substance is inside them. Stoves are affected by power cuts. After the cooking state reaches the optimal state, it will pause the cooking. If left on the stove, it will start to cook the food again. If still left unattended and not picked up off the stove, it will go on fire.

Oven:

Power: Yes

Trays can be placed in an oven. When inside the oven it will start a timer that increases the cooking state of the food. It will pause when fully cooked at the optimal time for a few seconds. Then, if not taken out of the oven, it will keep cooking until it burns and sets the oven on fire.

Deep Fryer:

Power: Yes

Fryer baskets can be placed in a deep fryer. When inside the fryer, a timer will increase the cooking state of the food inside the basket. It will pause when cooked at the optimal time, but if left unattended and not taken out of it, the timer will start again. If still left unattended it will set the Deep Fat Fryer on fire.

Blender:

Power: Yes

A variant of the bench that has a blender on it. The raw ingredients can be placed inside the blender. When placed, it will start a timer to increase the cooking state. It will briefly pause when it reaches an optimal state.

Freezer:

Power: Yes

Acts as storage for dishes. Preserves foods from being stolen by rats. Can be affected by power outages.

Trash:

Power: No

When interacted with, it will erase the cooking state of the plate/pot/pan the player is holding.

Food Crate:

Power: No

Stores one type of ingredient and when interacted with will give the player the ingredient into their hand.

Sink:

Power: No

Used to clean dirty plates. Pots can be interacted with to fill it up with water.

Items:

Pot
Frying Pan
Oven Tray
Plate
Bowl
Fryer Basket
Upgrade Hammer
Fire Extinguisher

Progression and Rewards

Players will progress slowly, building their restaurant up using money they earned from serving customers. This progression system will allow customers to have the freedom to build the restaurant with their vision. How reputation is handled will progress throughout the game. This entails how much reputation you gain and lose. At the start, the players will not lose much experience and will gain a standard amount. Later, it will become the case where reputation will become harder to earn but more impactful when you lose it. This allows players to make mistakes during the starting days and get familiar with the mechanics, and once they reach the later days, they will have that time to understand the game and will have more risk involved.

During gameplay, players will be rewarded with money and reputation. Money will be able to be spent on further progressing your restaurant's development. This gives the player the agency to want to complete the order as perfectly as possible, to be able to spend the money as they see fit.

Technical Design

Engine Choice

The game is planned to be developed in Godot 4.4. Godot supports multiplayer through its implementation of the client-server model. This is the engine our team has the most experience in, and it has the right features to be able to design and create the game. Multiplayer is going to be the hardest feature to implement, and having an engine that has tutorials about it and is simple to use is key for the development of the game.

Technical Requirements and Architecture

We will be using the client-server model to implement multiplayer. This model is quite easy to understand how it works and can be easily used in Godot. We will be using Godot's websockets to have connections between the client and the server. This model always allows us to have code only run through the server, minimizing players who may cheat. We aim to have the state of the game replicated around ~30ms. This is a good amount of time, so it's not every frame, but still enough that players have a smooth feeling experience.

As we are using Godot, we will be using GDScript to design the framework. With consideration of the scope of the game, we will need to decompose key systems into components that we can modularly reuse. An example of this would be having a 'timer' component; This could be used for appliances, a round timer, and orders.

Additionally, the use of 'Interfaces and 'Abstraction' can reduce redundant code. This can be implemented by having an 'Interactable' interface and an 'Item' class. This will make the architecture more manageable and help to scale with any additions to already existing systems.

Performance Targets and Optimization

Ideally the game will run at 120fps for the majority of the runtime. We can achieve this by various optimization techniques in various parts of systems, such as:

Object Pooling:

The use of will be crucial in optimizing the game's performance. We can use this for customers so we aren't constantly spawning and destroying them at runtime. We can spawn a couple of dozen at the start of a match, then reuse them when needed. We could also randomize their appearance when toggling them actively to give them a new customer. We can use the ingredients and limit how many of each ingredient can be on the map at one time.

Material Instances:

With the various meshes we will need, using material instances will lower the draw count significantly. We can further this optimization by having a master material from which we can create materials, so we can have different types of materials but still have them all instances of one master material. Furthermore, if needed, we can look into using custom primitive data.

Lighting:

We need to make sure that the lights we place don't overlap and minimize the use of point lights. Since the game takes place from a top-down perspective and the camera doesn't move, we don't need to worry about this too much, as the only real lighting will be from sunlight. Still will need to consider this if we want to add a day/night cycle to represent the Prep and Cooking phases.

Code:

Although on a case-by-case basis, we will need to limit the usage of [Event Tick] functions and rely on using signals instead. This helps decouple the code and optimization, updating information only when needed, not every frame. Additionally optimize our UI with usage of validation boxes to only re-render widgets when updated.

Platform considerations and deployment strategy

The game is planned to be released for Windows. There will be plans to support keyboard and controller. Once the game is completed and thoroughly tested, we plan to release it on Itch and possibly Steam. Near the end of development, we might consider having our game displayed at GDC. This will help us meet people in the games industry and get feedback from people who make games for a living.

Art and Audio Direction

Visual Style and Art Direction

We want the game to be in a low-poly style, with colors feeling vibrant and cozy. This style would suit the overall feel of the game and help with optimization and work within the time frame we have. The majority of these assets and animations we will be making ourselves. The low poly style has been used for various games within this genre, so following this similar choice will bring familiarity to players, but with our own look with the assets.

Audio Design and Music Style

During the prep phase, we want to have more relaxed, chill music to juxtapose with the hectic serving phases. This change in beat will bring the player back down and prepare for the spike in chaos when the serving starts once again.

The serving phase will initially start with the same music from the prep phase. Gradually, this will slowly transform into more upbeat music to match the chaos in the kitchen. Ideally, we will want to have the tempo of the music match how many customers are ordering at that given time.

Having snappy sound effects for the varying interactions in the game will give players feedback on what they're doing and immerse themselves more in the restaurant life. With a team member currently partaking in an audio course, hopefully some of the sounds we can make ourselves; The rest we will likely find externally online.

UI/UX

UI will be limited during the bulk of the gameplay. This is the short list of main areas we will have UI

- Warnings about incoming sabotage
- Your restaurants money
- Visuals for cooking state
- Customers orders
- Time for each round
- Brief information about upgrades

This list is excluding menus such as the main menu, pause menu, settings, etc, as these are more of the stock standard for games to have. We want the overall design of the UI to be fun and easy to use; Simple to use and navigate.

Level Design

Game world structure

The game world will take place in varying locations. Since the game is exclusively multiplayer, the game world won't be in a linear fashion. We want there to be enough variations so players don't get burned out playing in the same map. Each map, we want there to be some special interaction that makes it more than just an environment. Players will be able to choose from multiple kitchen layouts before the match. This will give each game a more unique feel and make each game feel fresh. This is the list of the planned maps and the interaction within them.

Maps	
<i>Location</i>	<i>Interaction</i>
City	N/A
Pirate Ship	Shoot cannons at each other
Swamp	Flies come and steal items on benches
Backyard	Can enter other backyard through fence

Level Progression

Players at the beginning of each game will spawn into the restaurant layout chosen before the game. Progressing through the game, each restaurant will be generating money, and players can choose to upgrade their restaurant. This can be in the form of upgrading existing appliances or building new ones. Players will be able to edit where appliances are. This control over their restaurant will serve as the progression as they build their restaurant from the ground up.

Production Plan

Development Phase and milestones

Development has been decomposed into small milestones organized into phases that will guide our production.

Pre-Production:

Goals:

- Get the team all synced up with the concept
- Sort out roles
- Decomposing the project

Tasks:

- Layout architecture
- Github
- Task & role allocation

MVP:

Goals:

- Core gameplay implemented
- Exploring multiplayer implementation
- See if the game loop works

Tasks:

- Reputation system
- Player controller
- Currency system
- Order System
- Customers

Final:

Goals:

- Polish

Tasks:

- Assets
- VFX
- Sounds

Resource management and team structure

The assets for this project are planned to be created by us. This is so we have our unique look for the game and to minimize the worry of licenses for assets. If we do find ourselves in a time constraint, it will open the possibility of using external assets, but to a minimal amount. This ideology is the same as the 2D assets and the sound design.

With a team of seven, we are planning to break ourselves into roles. These roles will mainly be using the animal system initially, then during development will start leaning into what is more suited at the time. We have five programmers that will be working on separate systems and two game designers that will work on modelling, animation, and level design.

Risk assessment and mitigation strategies

There are a few ways we are going to handle our version control to mitigate merge conflicts.

- Each team member's work/features are on different branches
- Approval for merges from team members
- Consistent weekly merges

Enforcing this workflow within our team will hopefully lessen the risk of merge conflicts. This will ensure that we are all working on separate systems, and if we do need to edit, we can communicate about it with each other. Approvals for merges will strengthen the quality of code we push and ensure broken code is pushed into the main branch. We will try to incorporate everyone's merging during our meetings so we all can work with the latest changes, and we can catch merge conflicts early.

Version control and project management strategy

We will be using Git for our version control. It's free and used in the wider industry, so getting more experience and becoming proficient in using it is ideal. Initially, we will be splitting our team into our separate branches to work on building the MVP. Later during development, we will hopefully switch to a more feature-based branch as multiple members will likely be working on the same task.

For project management, we will be either using GitHub or Trello to lay out tasks. This will be a decision discussed by the team. This will help manage, tie, and see what needs doing and what everyone is working on. Additionally, we will be using Discord to efficiently communicate with each other. This is so we can reach out for help and, in general, talk about the project outside of assigned class times and meetings for the week.

Testing and quality assurance

Throughout development, we will be asking peers to playtest and quality assure the mechanics, system, and overall how the game plays and feels. Especially after we have developed a strong MVP, we would want to get playtest feedback every other day from various people. Their feedback would inform what we needed to work on that week. With the nature of so many systems and mechanics working harmoniously together, having them tested regularly will be beneficial for the game's development.